

Spring

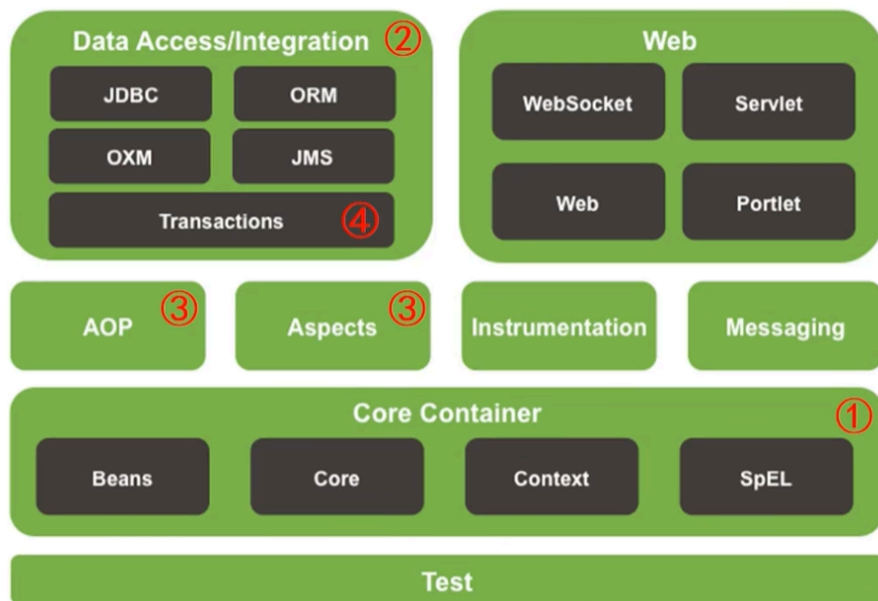
鸣谢：黑马程序员



黑马程序员SSM框架教程_Spring+SpringMVC+Maven高级+SpringBo...

UP 黑马程序员 · 2022-4-20

Spring Framework系统架构



- Data Access: 数据访问
- Data Integration: 数据集成
- Web: Web开发
- AOP: 面向切面编程
- Aspects: AOP思想实现
- Core Container: 核心容器
- Test: 单元测试与集成测试

• Spring

一、核心容器

1. IoC/DI

1.1 IoC

1.2 IoC 容器

1.3 Bean

P1 Bean 的作用范围(scope)

P2 实例化 Bean 的方式

P3 Bean 的生命周期及控制

P4 Bean 的销毁时机

1.4 DI

P1 概述

P2 依赖注入方式

P3 如何选择依赖注入方式?

P4 依赖自动装配

P5 集合注入

P6 加载 properties 文件

2.容器基本操作

2.1 创建容器

P1 加载配置文件的两种方式

P2 加载多个配置文件

2.2 获取 bean

2.3 容器类层次结构

2.4 BeanFactory

3.注解开发

3.1 注解开发定义 bean

P1 步骤

P2 Spring提供的三个衍生注解

3.2 纯注解开发 (Spring3.0+)

3.3 控制 Bean 的作用范围与生命周期

3.4 依赖注入——自动装配

P1 使用 @Autowired 注解开启按类型自动装配模式

P2 使用 @Qualifier 注解按指定名称装配 bean

P3 使用 @Value 注解注入简单类型数据

P4 使用 @PropertySource 注解加载属性文件

3.5 管理第三方 bean

P1 使用 @Bean 注解来配置第三方 bean

P2 为第三方 bean 进行依赖注入

3.6 @Component 和 @Bean 的区别

P1 概述

P2 对比表

二、数据访问与集成

1. Spring 整合 MyBatis

2.1 mybatis-config.xml

2.2 代码重构

2. Spring 整合 JUnit

三、AOP

1.概述

2.核心概念

3.实现步骤

4. AOP 工作流程

5.切入点表达式

5.1 标准格式

5.2 通配符

5.3 书写技巧

6.通知类型

6.1 前置通知

6.2 后置通知

6.3 环绕通知（重点）

6.4 返回后通知（了解）

6.5 抛出异常后通知（了解）

7.从通知中获取数据

7.1 获取参数

7.2 获取返回值

7.3 获取异常

四、事务

1.概述

2.Spring事务

3.开启事务的步骤——案例：银行转账

3.1 在业务层接口上添加Spring事务管理

3.2 设置事务管理器

3.3 开启注解式事务驱动

4.Spring事务角色

5.Spring事务相关配置

一、核心容器

1. IoC/DI

1.1 IoC

- **IoC(Inversion of Control)**：控制反转。
- **核心思想**：将对象的创建控制权由程序内部转移到**外部**。也就是使用对象时，在程序中不要主动使用 `new` 产生对象，而是转换为由外部提供对象。

1.2 IoC 容器

- Spring提供了一个容器用来充当 **IoC** 思想中的**外部**，称为 **IoC** 容器。
-

1.3 Bean

Important

IoC 容器负责对象的创建、初始化等一系列工作，被创建或被管理的对象在 IoC 容器中统称为 Bean。

P1 Bean 的作用范围(scope)

- 默认是单例。
- 适合交给容器管理的 Bean：接口层对象、业务层对象、数据层对象、工具对象；
- 不适合交给容器管理的 Bean：封装实体的域对象。

P2 实例化 Bean 的方式

- 常用：提供无参构造器（`private` 也可）。
- 使用静态工厂。
- 使用实例工厂。
- 重点：使用 `FactoryBean`。（方式三的变种）

P3 Bean 的生命周期及控制

- Bean 的生命周期：
 - Step1: 初始化容器。
 1. 创建对象（内存分配）
 2. 执行构造器
 3. 执行属性注入（调用 `setter`）
 4. 执行 `bean` 初始化方法
 - Step2: 使用 `bean`。
 - Step3: 关闭/销毁容器。
- Bean 的生命周期控制：
 - 控制方式一：

- 提供生命周期控制方法

```
public class BookDaoImpl implements BookDao {
    public void save() {
        System.out.println("book dao save ...");
    }
    public void init(){
        System.out.println("book init ...");
    }
    public void destory(){
        System.out.println("book destory ...");
    }
}
```

- 配置生命周期控制方法

```
<bean id="bookDao" class="com.itheima.dao.impl.BookDaoImpl" init-method="init" destroy-method="destory"/>
```

◦ 控制方式二：接口控制

- 实现InitializingBean, DisposableBean接口

```
public class BookServiceImpl implements BookService , InitializingBean, DisposableBean {
    public void save() {
        System.out.println("book service save ...");
    }
    public void afterPropertiesSet() throws Exception {
        System.out.println("afterPropertiesSet");
    }
    public void destroy() throws Exception {
        System.out.println("destroy");
    }
}
```

P4 Bean 的销毁时机

- 容器关闭前触发 Bean 的销毁。
 - 关闭容器的方式：
 - 手动关闭：调用 ConfigurableApplicationContext 接口的 close() 方法。
 - 注册关闭钩子，先关闭容器再退出JVM：调用 ConfigurableApplicationContext 接口的 registerShutdownHook() 方法。
-

1.4 DI

P1 概述

- **DI(Dependency Injection)**：依赖注入。
- **核心思想**：在 **IoC** 容器中建立 **Bean** 与 **Bean** 之间的依赖关系。



- 向一个类中传递的数据类型：
 - 简单类型：基本数据类型 + **String** --> **<value>**
 - 引用类型 --> **<ref>**

P2 依赖注入方式

- **setter 注入**：
 - 简单类型
 - 引用类型
- **构造器注入**：
 - 简单类型
 - 引用类型

P3 如何选择依赖注入方式？

- 具有强制依赖关系的使用构造器注入，因为使用 **setter** 注入有概率没有进行注入，导致 **null** 对象出现。
- 具有可选依赖关系的使用 **setter** 注入，灵活性强。
- Spring框架推荐使用构造器注入，第三方框架内部大多采用构造器注入的形式进行数据初始化，相对严谨。
- 如有必要可以两者同时使用：

- 使用构造器注入完成强制依赖关系的注入；
- 使用 `setter` 注入完成可选依赖关系的注入。
- 实际开发过程中根据实际情况分析，如果受控对象没有提供 `setter`，则必须使用构造器注入。
- 自己开发的模块**推荐**使用 `setter` 注入。

P4 依赖自动装配

- **含义：** `IoC` 容器根据 `bean` 所依赖的资源在容器中自动查找并注入到 `bean` 中。
- **自动装配方式：**
 - 按类型（**常用**）
 - 按名称
 - 按构造器
 - 不启用自动装配
- **注意事项：**
 - 自动装配仅用于引用类型的依赖注入，无法对简单类型进行依赖注入。
 - 使用按类型装配(`byType`)时，必须保证容器中相同类型的 `bean` 只有一个，开发中**推荐**使用这种方式进行自动装配。
 - 使用按名称装配(`byName`)时，必须保证容器中具有指定名称的 `bean`（即 `setter` 方法的名字去掉 `set` 后的名称），因为变量名与配置高度耦合，故不推荐使用。
 - 自动装配的优先级低于 `setter` 注入和构造器注入，同时出现时自动装配失效。

P5 集合注入

- `BookDaoImpl`：

```
1 package com.zsh.dao.impl;
2
3 import com.zsh.dao.BookDao;
4 import java.util.*;
5
6 public class BookDaoImpl implements BookDao {
7
```

```

8     private int[] zshArray;
9     private List<String> zshList;
10    private Set<String> zshSet;
11    private Map<String,String> zshMap;
12    private Properties zshProperties;
13
14    public void setZshArray(int[] zshArray) {
15        this.zshArray = zshArray;
16    }
17    public void setZshList(List<String> zshList) {
18        this.zshList = zshList;
19    }
20    public void setZshSet(Set<String> zshSet) {
21        this.zshSet = zshSet;
22    }
23    public void setZshMap(Map<String, String> zshMap) {
24        this.zshMap = zshMap;
25    }
26    public void setZshProperties(Properties zshProperties) {
27        this.zshProperties = zshProperties;
28    }
29
30    @Override
31    public void save() {
32        System.out.println("book dao save ...");
33        System.out.println("traverse Array: "+
Arrays.toString(zshArray));
34        System.out.println("traverse List: "+ zshList);
35        System.out.println("traverse Set: "+ zshSet);
36        System.out.println("traverse Map: "+ zshMap);
37        System.out.println("traverse Properties: "+ zshProperties);
38    }
39 }

```

- applicationContext.xml

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
5
6     <bean id="bookDao" class="com.zsh.dao.impl.BookDaoImpl">
7         <property name="zshArray">

```

```
8         <array>
9             <value>100</value>
10            <value>200</value>
11            <value>300</value>
12        </array>
13    </property>
14
15    <property name="zshList">
16        <list>
17            <value>zsh</value>
18            <value>zjl</value>
19            <value>zxj</value>
20            <value>zxj</value>
21        </list>
22    </property>
23
24    <property name="zshSet">
25        <set>
26            <value>apple</value>
27            <value>banana</value>
28            <value>strawberry</value>
29            <value>strawberry</value>
30            <!-- 由于Set元素不可重复，故会自动过滤2个重复的strawberry-->
31        </set>
32    </property>
33
34    <property name="zshMap">
35        <map>
36            <entry key="country" value="China"/>
37            <entry key="province" value="Guangdong"/>
38            <entry key="city" value="Swatow"/>
39        </map>
40    </property>
41
42    <property name="zshProperties">
43        <props>
44            <prop key="name">zsh</prop>
45            <prop key="sex">male</prop>
46            <prop key="height">175</prop>
47        </props>
48    </property>
49 </bean>
50
51 </beans>
```

P6 加载 properties 文件

加载properties文件

- 开启context命名空间

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           http://www.springframework.org/schema/context/spring-context.xsd">
</beans>
```

- 使用context命名空间，加载指定properties文件

```
<context:property-placeholder location="jdbc.properties"/>
```

- 使用\${}读取加载的属性值

```
<property name="username" value="${jdbc.username}"/>
```

- 不加载系统属性

```
<context:property-placeholder location="jdbc.properties" system-properties-mode="NEVER"/>
```

- 加载多个properties文件

```
<context:property-placeholder location="jdbc.properties,msg.properties"/>
```

- 加载所有properties文件

```
<context:property-placeholder location="*.properties"/>
```

- 加载properties文件**标准格式**

```
<context:property-placeholder location="classpath:*.properties"/>
```

- 从类路径或jar包中搜索并加载properties文件

```
<context:property-placeholder location="classpath*:*.properties"/>
```

2. 容器基本操作

2.1 创建容器

P1 加载配置文件的两种方式

- 通过类路径: `ApplicationContext context = new ClassPathXmlApplicationContext("????.xml");`
- 通过文件路径: `ApplicationContext context = new FileSystemXmlApplicationContext("????.xml的绝对路径或本工程下的相对路径");`

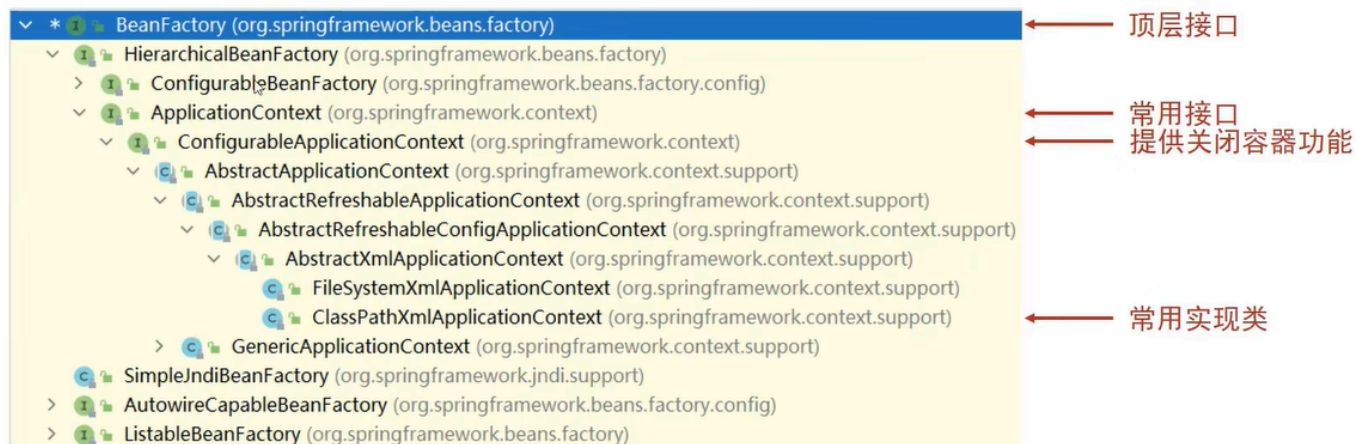
P2 加载多个配置文件

```
ApplicationContext context = new ClassPathXmlApplicationContext("bean1.xml",  
"bean2.xml");
```

2.2 获取 bean

- 使用 bean 名称获取: `BookDao bookDao = (BookDao) context.getBean("bookDao")`
- 使用 bean 名称获取并指定 bean 类型: `BookDao bookDao = context.getBean("bookDao", BookDao.class)`
- 使用 bean 类型获取 (注意该类型的 bean 在配置文件中必须是唯一的): `BookDao bookDao = context.getBean(BookDao.class)`

2.3 容器类层次结构



2.4 BeanFactory

BeanFactory初始化



- 类路径加载配置文件

```
Resource resources = new ClassPathResource("applicationContext.xml");
BeanFactory bf = new XmlBeanFactory(resources);
BookDao bookDao = bf.getBean("bookDao", BookDao.class);
bookDao.save();
```

- BeanFactory创建完毕后，所有的bean均为延迟加载

3.注解开发

3.1 注解开发定义 bean

P1 步骤

Step1: 使用 **@Component** 定义 **bean**。

```
1 @Component("bookDao")
2 public class BookDaoImpl implements BookDao {
3 }
4
5 @Component
6 public class BookServiceImpl implements BookService {
7 }
```

Step2: 核心配置文件中通过组件扫描加载 `bean`。

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:context="http://www.springframework.org/schema/context"
4       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5       xsi:schemaLocation="
6         http://www.springframework.org/schema/beans
7         http://www.springframework.org/schema/beans/spring-beans.xsd
8         http://www.springframework.org/schema/context
9         http://www.springframework.org/schema/context/spring-context.xsd
10    ">
11     <context:component-scan base-package="com.zsh"/>
12
13 </beans>
```

P2 Spring提供的三个衍生注解

💡 Tip

功能和 `@Component` 一模一样，只是增强了可读性。

- `@Controller`：用于接口层 `bean` 定义。
- `@Service`：用于业务层 `bean` 定义。
- `@Repository`：用于数据层 `bean` 定义。

3.2 纯注解开发 (Spring3.0+)

创建一个Java类 `SpringConfig` 来代替核心配置文件:

```
1 package com.zsh.config;
2
3 import org.springframework.context.annotation.ComponentScan;
4 import org.springframework.context.annotation.Configuration;
5
6 @Configuration// 代表这是个Spring配置类
7 @ComponentScan("com.zsh")// 指定扫描路径, 此注解只能添加一次, 多个数据需用数组
   形式, 如@ComponentScan({"com.zsh.dao", "com.zsh.service"})
8 public class SpringConfig {
9 }
```

重新编写测试类:

```
1 package com.zsh;
2
3 import com.zsh.config.SpringConfig;
4 import com.zsh.dao.BookDao;
5 import com.zsh.service.BookService;
6 import
   org.springframework.context.annotation.AnnotationConfigApplicationConte
   xt;
7
8 public class AppByPureAnnotation {
9     public static void main(String[] args) {
10         ApplicationContext context=new
   AnnotationConfigApplicationContext(SpringConfig.class);
11
12         BookDao bookDao=(BookDao) context.getBean("bookDao");
13         System.out.println(bookDao);
14         BookService bookService=context.getBean(BookService.class);
15         System.out.println(bookService);
16     }
17 }
```

3.3 控制 Bean 的作用范围与生命周期

bean作用范围

- 使用@Scope定义bean作用范围

```
@Repository
@Scope("singleton")
public class BookDaoImpl implements BookDao {
}
```

bean生命周期

- 使用@PostConstruct、@PreDestroy定义bean生命周期

```
@Repository
@Scope("singleton")
public class BookDaoImpl implements BookDao {
    public BookDaoImpl() {
        System.out.println("book dao constructor ...");
    }
    @PostConstruct
    public void init(){
        System.out.println("book init ...");
    }
    @PreDestroy
    public void destroy(){
        System.out.println("book destroy ...");
    }
}
```

3.4 依赖注入——自动装配

P1 使用 `@Autowired` 注解开启按类型自动装配模式

- 使用`@Autowired`注解开启自动装配模式（按类型）

```
@Service
public class BookServiceImpl implements BookService {
    @Autowired
    private BookDao bookDao;
    public void setBookDao(BookDao bookDao) {
        this.bookDao = bookDao;
    }
    public void save() {
        System.out.println("book service save ...");
        bookDao.save();
    }
}
```

- 注意：自动装配基于反射设计创建对象并暴力反射对应属性为私有属性初始化数据，因此无需提供setter方法
- 注意：自动装配建议使用无参构造方法创建对象（默认），如果不提供对应构造方法，请提供唯一的构造方法

P2 使用 `@Qualifier` 注解按指定名称装配 bean

- 使用`@Qualifier`注解开启指定名称装配bean

```
@Service
public class BookServiceImpl implements BookService {
    @Autowired
    @Qualifier("bookDao")
    private BookDao bookDao;
}
```

- 注意：`@Qualifier`注解无法单独使用，必须配合`@Autowired`注解使用

P3 使用 `@Value` 注解注入简单类型数据

- 使用`@Value`实现简单类型注入

```
@Repository("bookDao")
public class BookDaoImpl implements BookDao {
    @Value("100")
    private String connectionNum;
}
```

P4 使用 `@PropertySource` 注解加载属性文件

加载properties文件

- 使用`@PropertySource`注解加载properties文件

```
@Configuration
@ComponentScan("com.itheima")
@PropertySource("classpath:jdbc.properties")
public class SpringConfig {
}
```

- 注意：路径仅支持单一文件配置，多文件请使用数组格式配置，不允许使用通配符*

3.5 管理第三方 bean

P1 使用 `@Bean` 注解来配置第三方 bean

- `SpringConfig`:

```

1 package com.zsh.config;
2
3 import org.springframework.context.annotation.Configuration;
4 import org.springframework.context.annotation.Import;
5
6 @Configuration// 代表这是个Spring配置类
7 @Import(JdbcConfig.class)// 导入配置
8 public class SpringConfig {
9 }

```

- **JdbcConfig**: (使用独立的配置类进行解耦合)

```

1 package com.zsh.config;
2
3 import com.alibaba.druid.pool.DruidDataSource;
4 import org.springframework.context.annotation.Bean;
5 import javax.sql.DataSource;
6
7 public class JdbcConfig {
8
9     // 1.定义一个方法来获得要管理的第三方bean
10    // 2.添加@Bean表示该方法的返回值是一个bean
11    @Bean
12    public DataSource getDataSource(){
13        DruidDataSource dataSource=new DruidDataSource();
14        dataSource.setDriverClassName("com.mysql.jdbc.Driver");
15        dataSource.setUrl("jdbc:mysql://localhost:3306/spring_db");
16        dataSource.setUsername("root");
17        dataSource.setPassword("123456");
18        return dataSource;
19    }
20 }

```

P2 为第三方 **bean** 进行依赖注入

- 简单类型依赖注入: **设置成员变量。**

```

1 public class JdbcConfig {
2
3     @Value("com.mysql.jdbc.Driver")

```

```

4     private String driver;
5     @Value("jdbc:mysql://localhost:3306/spring_db")
6     private String url;
7     @Value("root")
8     private String username;
9     @Value("123456")
10    private String password;
11
12    @Bean
13    public DataSource getDataSource(){
14        DruidDataSource dataSource=new DruidDataSource();
15        dataSource.setDriverClassName(driver);
16        dataSource.setUrl(url);
17        dataSource.setUsername(username);
18        dataSource.setPassword(password);
19        return dataSource;
20    }
21 }

```

- 引用类型依赖注入：只需要为带有 `@Bean` 的方法设置形参即可，容器会根据类型**自动装配**对象。

💡 Tip

`BookDaoImpl` 这个类记得加上 `@Repository` 注解，容器才可以自动装配。

```

1 public class JdbcConfig {
2
3     @Bean
4     public DataSource getDataSource(BookDao bookDao){// 容器会自动装配
5         bookDao(byType)
6         System.out.println(bookDao);
7         DruidDataSource dataSource=new DruidDataSource();
8         return dataSource;
9     }
10 }

```

3.6 @Component 和 @Bean 的区别

P1 概述

- @Component 是类级别注解，表示把这个类交给Spring容器管理。
- @Bean 是方法级别注解，表示把这个方法返回的对象交给Spring容器管理。
- 两者都能创建Spring容器中的 Bean，但用途完全不同。

P2 对比表

对比项	@Component	@Bean
注解位置	类	方法（通常在 @Configuration 类中）
Bean 创建方式	Spring自动扫描并实例化（反射调用无参构造器）	方法返回对象由Spring注册
依赖注入	通过 @Autowired、@Value 等注解进行依赖注入	方法形参自动依赖注入
适用场景	自己编写的类，且可被Spring扫描到	第三方类、需要定制构造/初始化逻辑的 Bean
配置方式	需配合 @ComponentScan 扫描包路径	不需要组件扫描，只要执行配置类即可
灵活性	相对固定，依赖无参构造器	高度灵活，可在方法中写任意 Java 代码控制实例化

二、数据访问与集成

1. Spring 整合 MyBatis

2.1 mybatis-config.xml

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE configuration
3      PUBLIC "-//mybatis.org/DTD Config 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-config.dtd">
5  <configuration>
6      <!-- 初始化属性数据 -->
7      <properties resource="jdbc.properties"></properties>
8
9      <!-- 初始化类型别名 -->
10     <typeAliases>
11         <package name="com.zsh.domain"/>
12     </typeAliases>
13
14     <!-- 初始化数据源 -->
15     <environments default="mysql">
16         <environment id="mysql">
17             <transactionManager type="JDBC"></transactionManager>
18             <dataSource type="POOLED">
19                 <property name="driver" value="${jdbc.driver}"/>
20                 <property name="url" value="${jdbc.url}"/>
21                 <property name="username" value="${jdbc.username}"/>
22                 <property name="password" value="${jdbc.password}"/>
23             </dataSource>
24         </environment>
25     </environments>
26
27     <!-- 初始化映射配置：随业务而改变 -->
28     <mappers>
29         <package name="com.zsh.dao"/>
30     </mappers>
31 </configuration>
```

分析可知，要想让 Spring 整合 MyBatis，则要让 Spring 管理 SqlSessionFactory 对象。

2.2 代码重构

- SpringConfig:

```
1 package com.zsh.config;
2
3 import org.springframework.context.annotation.ComponentScan;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.context.annotation.Import;
6 import org.springframework.context.annotation.PropertySource;
7
8 @Configuration
9 @ComponentScan("com.zsh")
10 @PropertySource("classpath:jdbc.properties")
11 @Import({JdbcConfig.class, MybatisConfig.class})
12 public class SpringConfig {
13 }
```

- JdbcConfig:

```
1 package com.zsh.config;
2
3 import com.alibaba.druid.pool.DruidDataSource;
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.context.annotation.Bean;
6
7 import javax.sql.DataSource;
8
9 public class JdbcConfig {
10     @Value("${jdbc.driver}")
11     private String driver;
12     @Value("${jdbc.url}")
13     private String url;
14     @Value("${jdbc.username}")
15     private String username;
16     @Value("${jdbc.password}")
17     private String password;
18
19     @Bean
20     public DataSource dataSource(){
21         DruidDataSource dataSource=new DruidDataSource();
22         dataSource.setDriverClassName(driver);
23         dataSource.setUrl(url);
24         dataSource.setUsername(username);
```

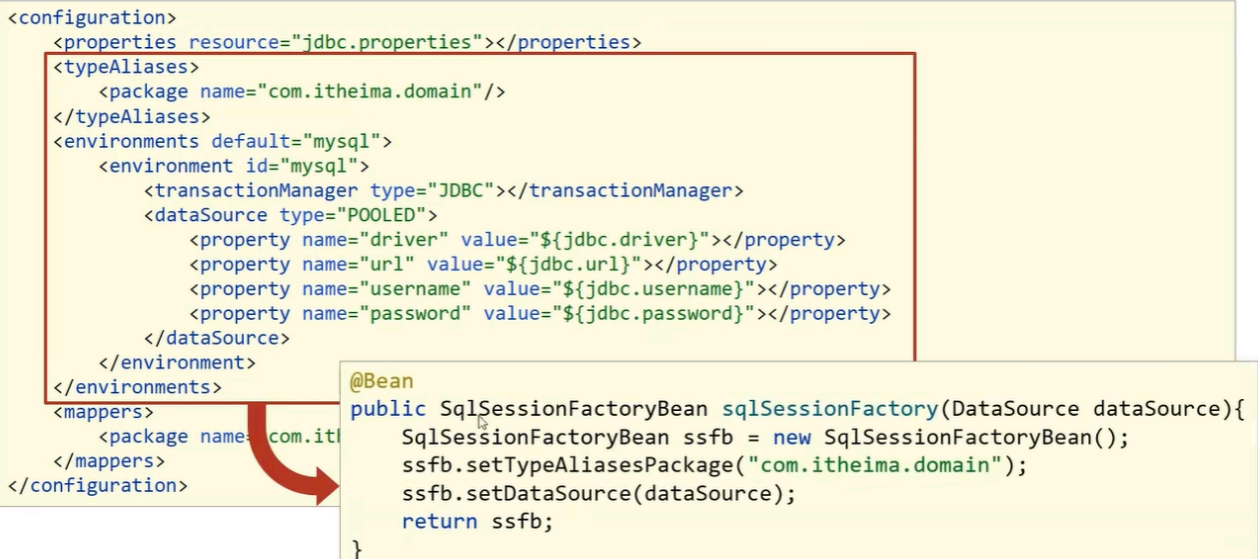
```

25         dataSource.setPassword(password);
26         return dataSource;
27     }
28 }

```

- MybatisConfig:

- 整合MyBatis



```

<configuration>
  <properties resource="jdbc.properties"></properties>
  <typeAliases>
    <package name="com.itheima.domain"/>
  </typeAliases>
  <environments default="mysql">
    <environment id="mysql">
      <transactionManager type="JDBC"></transactionManager>
      <dataSource type="POOLED">
        <property name="driver" value="${jdbc.driver}"></property>
        <property name="url" value="${jdbc.url}"></property>
        <property name="username" value="${jdbc.username}"></property>
        <property name="password" value="${jdbc.password}"></property>
      </dataSource>
    </environment>
  </environments>
  <mapppers>
    <package name="com.itheima.domain"></package>
  </mapppers>
</configuration>

```

```

@Bean
public SqlSessionFactoryBean sqlSessionFactory(DataSource dataSource){
    SqlSessionFactoryBean ssfb = new SqlSessionFactoryBean();
    ssfb.setTypeAliasesPackage("com.itheima.domain");
    ssfb.setDataSource(dataSource);
    return ssfb;
}

```

```

1 package com.zsh.config;
2
3 import org.apache.ibatis.session.SqlSessionFactory;
4 import org.mybatis.spring.SqlSessionFactoryBean;
5 import org.mybatis.spring.mapper.MapperScannerConfigurer;
6 import org.springframework.context.annotation.Bean;
7
8 import javax.sql.DataSource;
9
10 public class MybatisConfig {
11
12     @Bean
13     public SqlSessionFactoryBean sqlSessionFactoryBean(DataSource
14     dataSource){
15         SqlSessionFactoryBean factoryBean=new
16         SqlSessionFactoryBean();
17         factoryBean.setTypeAliasesPackage("com.zsh.domain");
18         factoryBean.setDataSource(dataSource);
19         return factoryBean;
20     }
21
22     @Bean
23     public MapperScannerConfigurer mapperScannerConfigurer(){

```

```
22         MapperScannerConfigurer configurer=new
    MapperScannerConfigurer();
23         configurer.setBasePackage("com.zsh.dao");
24         return configurer;
25     }
26 }
```

2. Spring 整合 JUnit

- AccountServiceTest:

```
1 package service;
2
3 import com.zsh.config.SpringConfig;
4 import com.zsh.service.AccountService;
5 import org.junit.Test;
6 import org.junit.runner.RunWith;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.test.context.ContextConfiguration;
9 import org.springframework.test.context.junit4.SpringJUnit4ClassRunner;
10
11 @RunWith(SpringJUnit4ClassRunner.class)
12 @ContextConfiguration(classes = SpringConfig.class)
13 public class AccountServiceTest {
14
15     @Autowired
16     private AccountService accountService;
17
18     @Test
19     public void testFindById(){
20         System.out.println(accountService.findById(2));
21     }
22 }
```

三、AOP

1.概述

- **AOP(Aspect-Oriented Programming)**：面向切面编程。是一种编程范式，指导开发者如何组织程序结构。
- **作用**：在不惊动原有设计的基础上为其进行功能增强（**无侵入式编程**）。

2.核心概念

Important

以下内容参考了Code With Sunil | Code Smarter, not harder在Medium平台发布的文章🔗[Mastering Spring AOP \(Aspect-Oriented Programming\): A Comprehensive Guide with Real-World Examples](#)

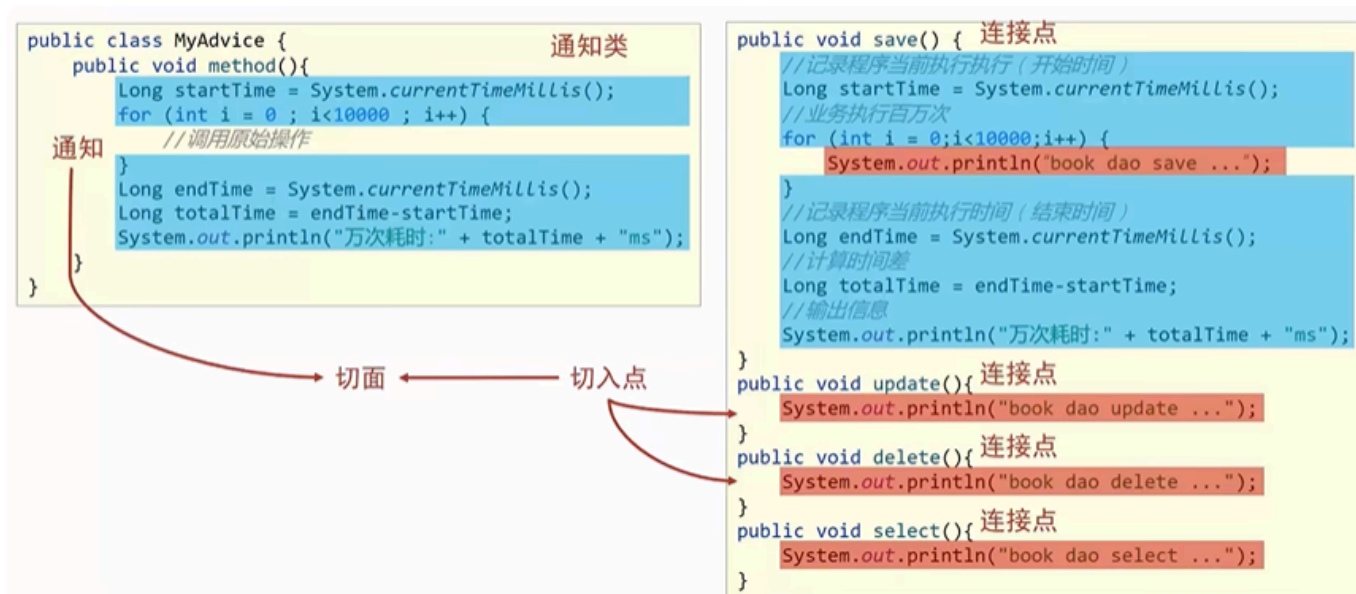
The diagram illustrates the AOP concept in Spring. It shows a 'Program Execution' flow with 'Join Points' marked by blue diamonds. A 'Pointcut' is defined, and 'Advice' is applied at the join points. The advice code is shown as:

```
// aspect code
if (...) {
} else {
...
}
```

@SunilMishra Happy Learning

- **连接点(Join Point)**：连接点是应用程序执行过程中的任意位置。

- 在Spring AOP中，连接点可以理解为方法的执行。
- **切入点(Pointcut)**: 切入点是一个匹配连接点的表达式，用于决定某个通知应该运行在哪些连接点上，它告诉Spring应该在何处应用你的切面。
 - 在Spring AOP中，一个切入点可以只描述一个具体方法，也可以匹配多个方法。
 - 一个具体方法: `com.zsh.dao` 包下的 `BookDao` 接口中的 `public void save()` 方法。
 - 匹配多个方法: 所有 `save` 方法，所有 `getXxx` 方法，所有 `xxxDao` 方法，所有带有一个形参的方法.....
 - **注意**: 切入点范围小于连接点范围，切入点包含于连接点中。
- **通知(Advice)**: 通知是你希望在切入点运行的代码，例如在方法运行之前先记录日志。
 - 在Spring AOP中，通知最终以方法的形式呈现。
- **通知类(Advice Class)**: 专门定义通知的类。
- **切面(Aspect)**: 切面是一个包含了多个类通用功能代码的模块，例如日志记录。它描述了通知与切入点的对应关系。
 - 在Spring AOP中，可以使用 `@Aspect` 注解或通过 `XML` 配置的方式来创建切面。



3. 实现步骤

1. 在 `pom.xml` 中导入 AOP 相关坐标。注意若已经导入了 `spring-context` 的坐标，就不用再导入 `spring-aop` 的坐标了，因为后者包含于前者。

```
1 <dependency>
2     <groupId>org.aspectj</groupId>
3     <artifactId>aspectjweaver</artifactId>
4     <version>自己选一个合适的版本号</version>
5 </dependency>
```

2. 制作连接点方法（原始操作，`Dao` 接口与实现类）。
3. 定义通知类，在通知类中编写共性功能，即通知。
4. 定义切入点，切入点依托于一个没有实际意义的方法进行，即无参数、无返回值、方法体无具体逻辑的方法。

```
1 public class MyAdvice {
2
3     @Pointcut("execution(void com.zsh.dao.BookDao.update())")
4     private void pointcutMethod(){}
5
6 }
```

5. 定义切面，绑定通知与切入点，同时将通知类交由Spring容器进行管理。

```
1 @Component
2 @Aspect// 用于创建切面
3 public class MyAdvice {
4
5     @Pointcut("execution(void com.zsh.dao.BookDao.update())")
6     private void pointcutMethod(){}
7
8     @Before("pointcutMethod()")
9     public void before(){
10         System.out.println(System.currentTimeMillis());
11     }
12 }
```

6. 在 `SpringConfig` 中开启Spring对 AOP 注解驱动的支持。

```
1 | @Configuration
2 | @ComponentScan("com.zsh")
3 | @EnableAspectJAutoProxy// 告诉Spring程序中存在用注解开发的AOP
4 | public class SpringConfig {
5 | }
```

4. AOP 工作流程

1. 启动Spring容器。
2. 读取所有绑定了通知的切入点。
3. 初始化 `bean`，判定 `bean` 对应的类中的方法是否匹配到任意切入点。
 - 若匹配失败，则正常创建 `bean` 对象。
 - 若匹配成功，则创建 `bean` 对象（称作原始对象或**目标对象**）的**代理对象**。
4. 获取 `bean`。
 - 若上一步匹配失败，则正常调用方法并执行，完成操作。
 - 若上一步匹配成功，则根据代理对象的运行模式运行原始方法与增强内容，完成操作。

5. 切入点表达式

5.1 标准格式

动作关键字(修饰符 返回值 包名.类/接口名.方法名(方法形参类型) 异常名)，其中修饰符、异常名可省略。

example: `execution(public User com.zsh.service.UserService.findById(int) RuntimeException)`。

动作关键字用于描述切入点的行为动作，`execution` 表示执行到指定切入点。

5.2 通配符

使用通配符可以快速描述切入点：

- `*`：单个任意符号，可以独立出现，也可以作为前缀或后缀匹配符出现。
 - `execution(public * com.zsh.*.UserService.find*(*))`：匹配 `com.zsh` 包下任意子包中的 `UserService` 中，所有以 `find` 为开头的，返回值任意且只有一个形参的方法。
- `..`：任意符号重复多次，可以独立出现，常用于简化包名与形参的书写。
 - `execution(public User com..UserService.findById(..))`：匹配 `com` 包下任意子包中的 `UserService` 中，所有名称为 `findById` 的方法，形参数量不限。
- `+`：专门用于匹配子类/实现类类型。
 - `execution(* *.*Service+.*(..))`：匹配任意包下的以 `Service` 为结尾的类/接口及其子类/实现类中的**所有方法**，即任意业务层接口。

5.3 书写技巧

- 所有代码按照标准规范开发，否则以下技巧全部失效。
 - 描述切入点通常描述接口，而不描述实现类，避免紧耦合。
 - 针对接口开发，修饰符均采用 `public` 描述。
 - 返回值类型对于**增、删、改**类使用精准类型加速匹配，对于**查询**类使用 `*` 通配符快速描述。
 - 包名书写尽量不使用 `..` 匹配，效率过低，建议使用 `*` 做单个包描述匹配，或精准匹配。
 - 若接口名/类名名称与模块相关，则采用 `*` 匹配，例如 `UserService` 书写成 `*Service`，绑定业务层接口名。
 - 方法名书写以动词进行精准匹配，名词采用 `*` 匹配，例如 `getById` 书写成 `getBy*`，`selectAll` 书写成 `selectAll`。
 - 参数规则较为复杂，根据业务方法灵活调整。
 - 通常不使用异常作为匹配规则。
-

6.通知类型

6.1 前置通知

- 名称：@Before
- 类型：方法注解
- 位置：通知方法定义上方
- 作用：设置当前通知方法与切入点之间的绑定关系，当前通知方法在原始切入点方法前运行
- 范例：

```
@Before("pt()")
public void before() {
    System.out.println("before advice ...");
}
```

- 相关属性：value（默认）：切入点方法名，格式为类名.方法名()

6.2 后置通知

- 名称：@After
- 类型：方法注解
- 位置：通知方法定义上方
- 作用：设置当前通知方法与切入点之间的绑定关系，当前通知方法在原始切入点方法后运行
- 范例：

```
@After("pt()")
public void after() {
    System.out.println("after advice ...");
}
```

- 相关属性：value（默认）：切入点方法名，格式为类名.方法名()

6.3 环绕通知（重点）

- 名称：@Around（重点，常用）
- 类型：**方法注解**
- 位置：通知方法定义上方
- 作用：设置当前通知方法与切入点之间的绑定关系，当前通知方法在原始切入点方法前后运行
- 范例：

```
@Around("pt()")
public Object around(ProceedingJoinPoint pjp) throws Throwable {
    System.out.println("around before advice ...");
    Object ret = pjp.proceed();
    System.out.println("around after advice ...");
    return ret;
}
```

● @Around注意事项

1. 环绕通知必须依赖形参ProceedingJoinPoint才能实现对原始方法的调用，进而实现原始方法调用前后同时添加通知
2. 通知中如果未使用ProceedingJoinPoint对原始方法进行调用将跳过原始方法的执行
3. 对原始方法的调用可以不接收返回值，通知方法设置成void即可，如果接收返回值，必须设定为Object类型
4. 原始方法的返回值如果是void类型，通知方法的返回值类型可以设置成void，也可以设置成Object
5. 由于无法预知原始方法运行后是否会抛出异常，因此环绕通知方法必须抛出Throwable对象

6.4 返回后通知（了解）

- 名称：@AfterReturning（了解）
- 类型：**方法注解**
- 位置：通知方法定义上方
- 作用：设置当前通知方法与切入点之间的绑定关系，当前通知方法在原始切入点方法正常执行完毕后运行
- 范例：

```
@AfterReturning("pt()")
public void afterReturning() {
    System.out.println("afterReturning advice ...");
}
```

- 相关属性：value（默认）：切入点方法名，格式为类名.方法名()

6.5 抛出异常后通知（了解）

- 名称：@AfterThrowing（了解）
- 类型：**方法注解**
- 位置：通知方法定义上方
- 作用：设置当前通知方法与切入点之间的绑定关系，当前通知方法在原始切入点方法运行抛出异常后执行
- 范例：

```
@AfterThrowing("pt()")
public void afterThrowing() {
    System.out.println("afterThrowing advice ...");
}
```

- 相关属性：value（默认）：切入点方法名，格式为类名.方法名()
-

7.从通知中获取数据

7.1 获取参数

- `JoinPoint`：描述了连接点方法的运行状态，可以获取到原始方法的调用参数（通过 `getArgs` 方法），适用于前置、后置、返回后、抛出异常后通知。
- `ProceedingJoinPoint`：是 `JoinPoint` 的子类，适用于环绕通知。

💡 Tip

如何通过 `ProceedingJoinPoint` 修改用户传入的参数？

```
1 @Around("pct()")
2     public Object around(ProceedingJoinPoint pjp) throws Throwable
3     {
4         // 获取用户传入的参数
5         Object[] args = pjp.getArgs();
6
7         // 修改参数
8         args[索引] = ...; // 填入具体修改逻辑
9
10        // 调用ProceedingJoinPoint提供的有参重载方法proceed(Object[]
11        args),
12        // 传入修改后的参数
13        Object ret=pjp.proceed(args);
14        return ret;
15    }
```

7.2 获取返回值

- 返回后通知：

```
1 @AfterReturning(value = "pct()",returning = "ret")// 指定ret用于接收原
2 始方法的返回值
3     public void afterReturning(Object ret){
4         System.out.println("afterReturning advice ...");
5         System.out.println("ret: "+ret);
6     }
```

- 环绕通知：

```
1 @Around("pct()")
2     public Object around(ProceedingJoinPoint pjp) throws Throwable {
3         Object ret=pjp.proceed();
4         return ret;
5     }
```

7.3 获取异常

- 抛出异常后通知：

```
1 | @AfterThrowing(value = "pct()",throwing = "t")// 指定t用于接收原始方法
   | 的异常
2 | public void afterThrowing(Throwable t){
3 |     System.out.println("afterThrowing advice ...");
4 |     System.out.println("exception: "+t);
5 | }
```

- 环绕通知：

```
1 | @Around("pct()")
2 | public Object around(ProceedingJoinPoint pjp) throws Throwable {
3 |     Object ret = null;
4 |     try {
5 |         ret = pjp.proceed();
6 |     } catch(Throwable t) {
7 |         t.printStackTrace();
8 |     }
9 |     return ret;
10 | }
```

四、事务

1.概述

Important

版权声明：以下内容摘选自CSDN博主「重生之我在成电转码」的原创文章，遵循CC 4.0 BY-SA版权协议，原文链接：https://blog.csdn.net/m0_69529796/article/details/146242971

事务(Transaction)是数据库操作的最小工作单元，保证多个SQL语句作为一个整体执行，要么全部成功，要么全部失败，确保数据的一致性和完整性。

例如，在银行转账的场景中：第一步：从A账户扣除100元；第二步：向B账户增加100元。

如果第一步执行成功，但第二步失败（比如数据库崩溃），账户A的钱就凭空少了，这会导致数据不一致。事务可以保证这两个操作要么都成功，要么都回滚（撤销），避免数据错误。

2.Spring事务

- **作用**：在数据层或业务层上，保障一系列的数据库操作要么同时成功，要么同时失败。
- Spring提供了一个接口 `PlatformTransactionManager`：

```
1 public interface PlatformTransactionManager{
2     void commit(TransactionStatus status) throws
    TransactionException; // 提交事务
3     void rollback(TransactionStatus status) throws
    TransactionException; // 回滚事务
4 }
```

- 针对 `JDBC` 的事物，Spring提供了上述接口的一个实现类 `DataSourceTransactionManager`。

3.开启事务的步骤——案例：银行转账

3.1 在业务层接口上添加Spring事务管理

⚠ Caution

Spring注解式事务退出添加在业务层接口，而非业务层实现类，以降低耦合。

`@Transactional` 可以添加在接口上，也可以添加在单个方法上。

```
1 | @Transactional// 开启事务
2 | public interface AccountService {
3 |     public void transfer(String out, String in, Double money);
4 | }
```

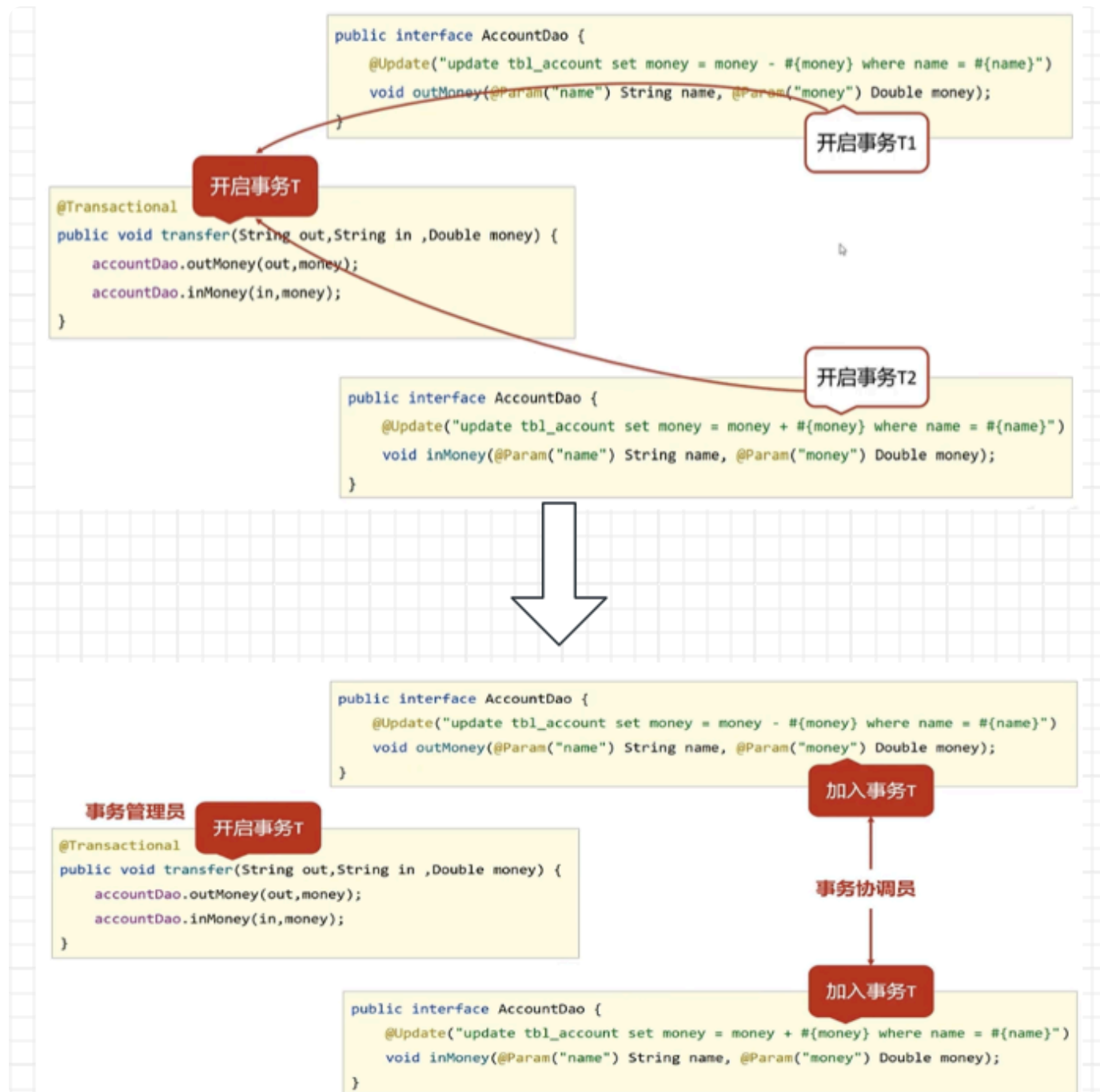
3.2 设置事务管理器

```
1 | public class JdbcConfig {
2 |     @Value("${jdbc.driver}")
3 |     private String driver;
4 |     @Value("${jdbc.url}")
5 |     private String url;
6 |     @Value("${jdbc.username}")
7 |     private String username;
8 |     @Value("${jdbc.password}")
9 |     private String password;
10 |
11 |     @Bean
12 |     public DataSource dataSource(){
13 |         DruidDataSource dataSource=new DruidDataSource();
14 |         dataSource.setDriverClassName(driver);
15 |         dataSource.setUrl(url);
16 |         dataSource.setUsername(username);
17 |         dataSource.setPassword(password);
18 |         return dataSource;
19 |     }
20 |
21 |     @Bean
22 |     // 事务管理器，引用类型DataSource通过形参自动注入
23 |     public PlatformTransactionManager transactionManager(DataSource
dataSource){
24 |         DataSourceTransactionManager manager=new
DataSourceTransactionManager();
25 |         manager.setDataSource(dataSource);
26 |         return manager;
27 |     }
28 | }
```

3.3 开启注解式事务驱动

```
1 @Configuration
2 @ComponentScan("com.zsh")
3 @PropertySource("classpath:jdbc.properties")
4 @Import({JdbcConfig.class, MybatisConfig.class})
5 @EnableTransactionManagement// 允许开启事务管理器
6 public class SpringConfig {
7 }
```

4.Spring事务角色



5.Spring事务相关配置

属性	作用	示例
readOnly	设置是否为只读事务	readOnly=true（只读事务）
timeout	设置事务超时时间	timeout=-1（永不超时）
rollbackFor	设置事务回滚异常（class）	rollbackFor={NullPointerException.class}
rollbackForClassName	设置事务回滚异常（String）	同上，但格式为字符串
noRollbackFor	设置事务不回滚异常（class）	noRollbackFor={NullPointerException.class}
noRollbackForClassName	设置事务不回滚异常（String）	同上，但格式为字符串
propagation	设置事务传播行为	见下

事务传播行为

传播属性	事务管理员	事务协调员
REQUIRED（默认）	开启T	加入T
	无	新建T2
REQUIRES_NEW	开启T	新建T2
	无	新建T2
SUPPORTS	开启T	加入T
	无	无
NOT_SUPPORTED	开启T	无
	无	无
MANDATORY	开启T	加入T
	无	ERROR
NEVER	开启T	ERROR
	无	无
NESTED	设置savePoint，一旦事务回滚，事务将回滚到savePoint处，交由客户响应提交/回滚	

